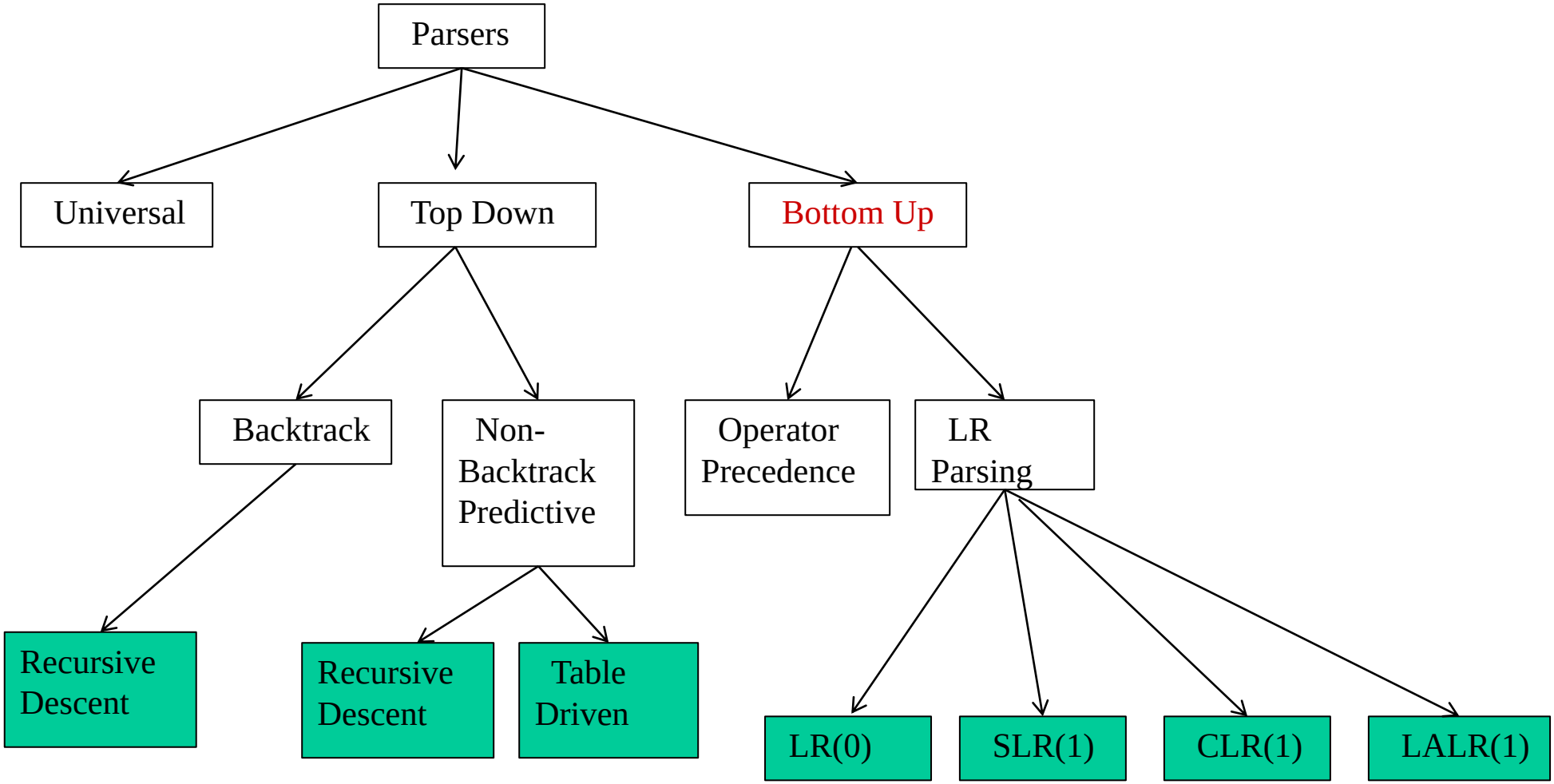
Types of Parsers



Bottom-Up Parsing

- A **bottom-up parser** creates the parse tree of the given input starting from leaves towards the root.
- A bottom-up parser tries to find the right-most derivation of the given input in the reverse order.

  $S \Rightarrow \ldots \Rightarrow \omega$ (the right-most derivation of ω)

  $\leftarrow$ (the bottom-up parser finds the right-most derivation in the reverse order)
- Bottom-up parsing is also known as **shift-reduce parsing** because its two main actions are shift and reduce.
  - At each shift action, the current symbol in the input string is pushed to a stack.
  - At each reduction step, the symbols at the top of the stack (this symbol sequence is the right side of a production) will replaced by the non-terminal at the left side of that production.
  - There are also two more actions: accept and error.

Shift-Reduce Parsing

- A shift-reduce parser tries to reduce the given input string into the starting symbol.

  a string ➔ the starting symbol

  reduced to

- At each reduction step, a substring of the input matching to the right side of a production rule is replaced by the non-terminal at the left side of that production rule.
- If the substring is chosen correctly, the right most derivation of that string is created in the reverse order.

Rightmost Derivation: $S \overset{*}{\underset{rm}{\Rightarrow}} \omega$

Shift-Reduce Parser finds: $S \underset{rm}{\Leftarrow} \ldots \underset{rm}{\Leftarrow} \omega$

Shift-Reduce Parsing -- Example

$S \rightarrow aABb$

$A \rightarrow aA \mid a$

$B \rightarrow bB \mid b$

input string: `aaabb`

`aaAbb`

`aAbb` $\Downarrow$ reduction

`aABb`

`S`

$$S \underset{rm}{\Rightarrow} aABb \underset{rm}{\Rightarrow} aAbb \underset{rm}{\Rightarrow} aaAbb \underset{rm}{\Rightarrow} aaabb$$

Right Sentential Forms

- How do we know which substring to be replaced at each reduction step?

Handle

- Informally, a **handle** of a string is a substring that matches the right side of a production rule.
  - But not every substring matches the right side of a production rule is handle
- Every string of symbols in the derivation is a **sentential form** and a **sentence *is a sentential*** form that has only terminal symbols
- A **handle** of a right sentential form γ ($\equiv \alpha\beta\omega$) is

  a production rule $A \rightarrow \beta$ and a position of γ

  where the string β may be found and replaced by A to produce

  the previous right-sentential form in a rightmost derivation of γ.

$$S \overset{*}{\underset{rm}{\Rightarrow}} \alpha A \omega \underset{rm}{\Rightarrow} \alpha\beta\omega$$

- If the grammar is unambiguous, then every right-sentential form of the grammar has exactly one handle.

Handle Pruning

- A right-most derivation in reverse can be obtained by **handle-pruning**.

$$S=\gamma_0 \underset{rm}{\Rightarrow} \gamma_1 \underset{rm}{\Rightarrow} \gamma_2 \underset{rm}{\Rightarrow} \cdots \underset{rm}{\Rightarrow} \gamma_{n-1} \underset{rm}{\Rightarrow} \gamma_n = \omega$$

input string

- Start from γ_n, find a handle $A_n \rightarrow \beta_n$ in γ_n, and replace β_n in by A_n to get γ_{n-1}.
- Then find a handle $A_{n-1} \rightarrow \beta_{n-1}$ in γ_{n-1}, and replace β_{n-1} in by A_{n-1} to get γ_{n-2}.
- Repeat this, until we reach S.

A Shift-Reduce Parser

$E \rightarrow E+T \mid T$

$T \rightarrow T*F \mid F$

$F \rightarrow (E) \mid id$

Right-Most Derivation of `id+id*id`

$E \Rightarrow E+T \Rightarrow E+T*F \Rightarrow E+T*id \Rightarrow E+F*id$

$\Rightarrow E+id*id \Rightarrow T+id*id \Rightarrow F+id*id \Rightarrow id+id*id$

Right-Most Sentential Form	Reducing Production (RMD in reverse)
`id+id*id`	$F \rightarrow id$
`F+id*id`	$T \rightarrow F$
`T+id*id`	$E \rightarrow T$
`E+id*id`	$F \rightarrow id$
`E+F*id`	$T \rightarrow F$
`E+T*id`	$F \rightarrow id$
`E+T*F`	$T \rightarrow T*F$
`E+T`	$E \rightarrow E+T$
`E`	

`Handles` are red and underlined in the right-sentential forms.

A Stack Implementation of A Shift-Reduce Parser

- There are four possible actions of a shift-reduce parser:

  1. **Shift** : The next input symbol is shifted onto the top of the stack.
  2. **Reduce**: Replace the handle on the top of the stack by the non-terminal.
  3. **Accept**: Successful completion of parsing.
  4. **Error**: Parser discovers a syntax error, and calls an error recovery routine.

- Initial stack just contains only the end-marker $ or starting state.
- The end of the input string is marked by the end-marker $.

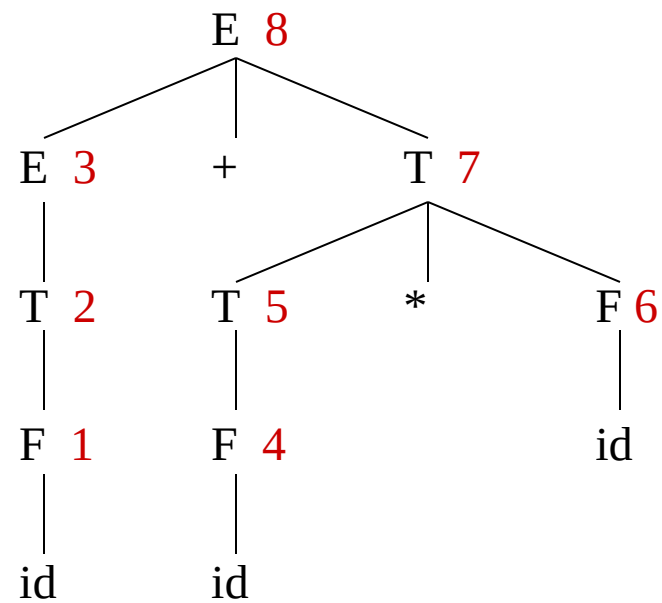
Shift-Reduce Parsing

```
push '$' onto the stack;
token = nextToken();
repeat
    if (there is a handle  A::=b on top of the stack){
        reduce b  to  A; /* reduce */
        pop b off the stack;
        push A onto the stack;
    } else {/* shift */
        shift token onto the stack;
        token = nextToken();
    }
until (top of stack is S and token is eof)
```

A Stack Implementation of A Shift-Reduce Parser

Stack	**Input**	**Action**
$	id+id*id$	shift
$id	+id*id$	reduce by F → id
$F	+id*id$	reduce by T → F
$T	+id*id$	reduce by E → T
$E	+id*id$	shift
$E+	id*id$	shift
$E+id	*id$	reduce by F → id
$E+F	*id$	reduce by T → F
$E+T	*id$	shift
$E+T*	id$	shift
$E+T*id	$	reduce by F → id
$E+T*F	$	reduce by T → T*F
$E+T	$	reduce by E → E+T
$E	$	accept

Parse Tree

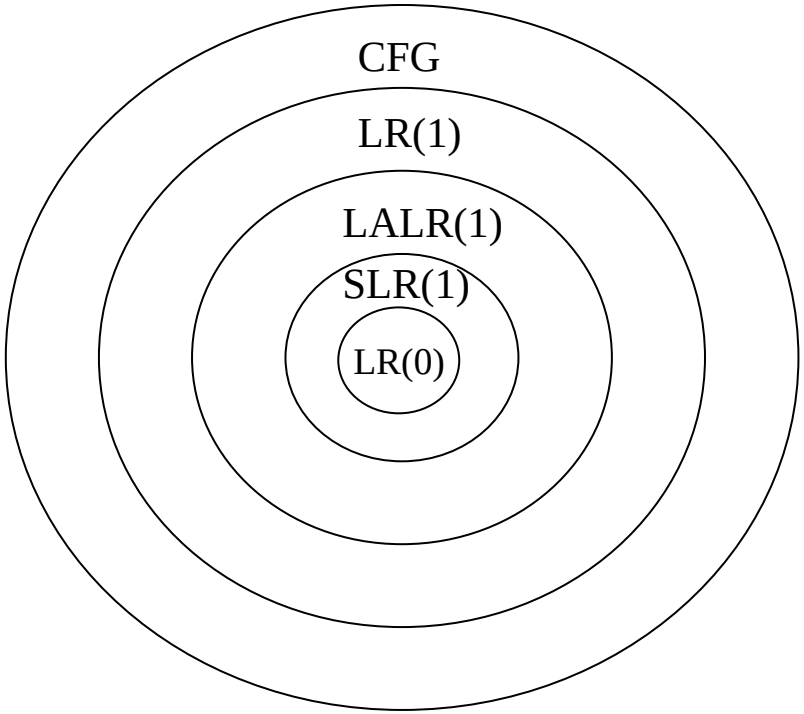


Shift-Reduce Parsers

- There are two main categories of shift-reduce parsers

1. Operator-Precedence Parser

– simple, but only a small class of grammars.



2. LR-Parsers

– Main: LR(0), LR(1)

– Some variations SLR and LALR(1)

– LR(0) Items – LR(0),SLR(1)

– LR(1) Items – LR(1), LALR(1)

LR Parsers

- The most powerful shift-reduce parsing (yet efficient) is:

LR(k) parsing.

left to right scanning

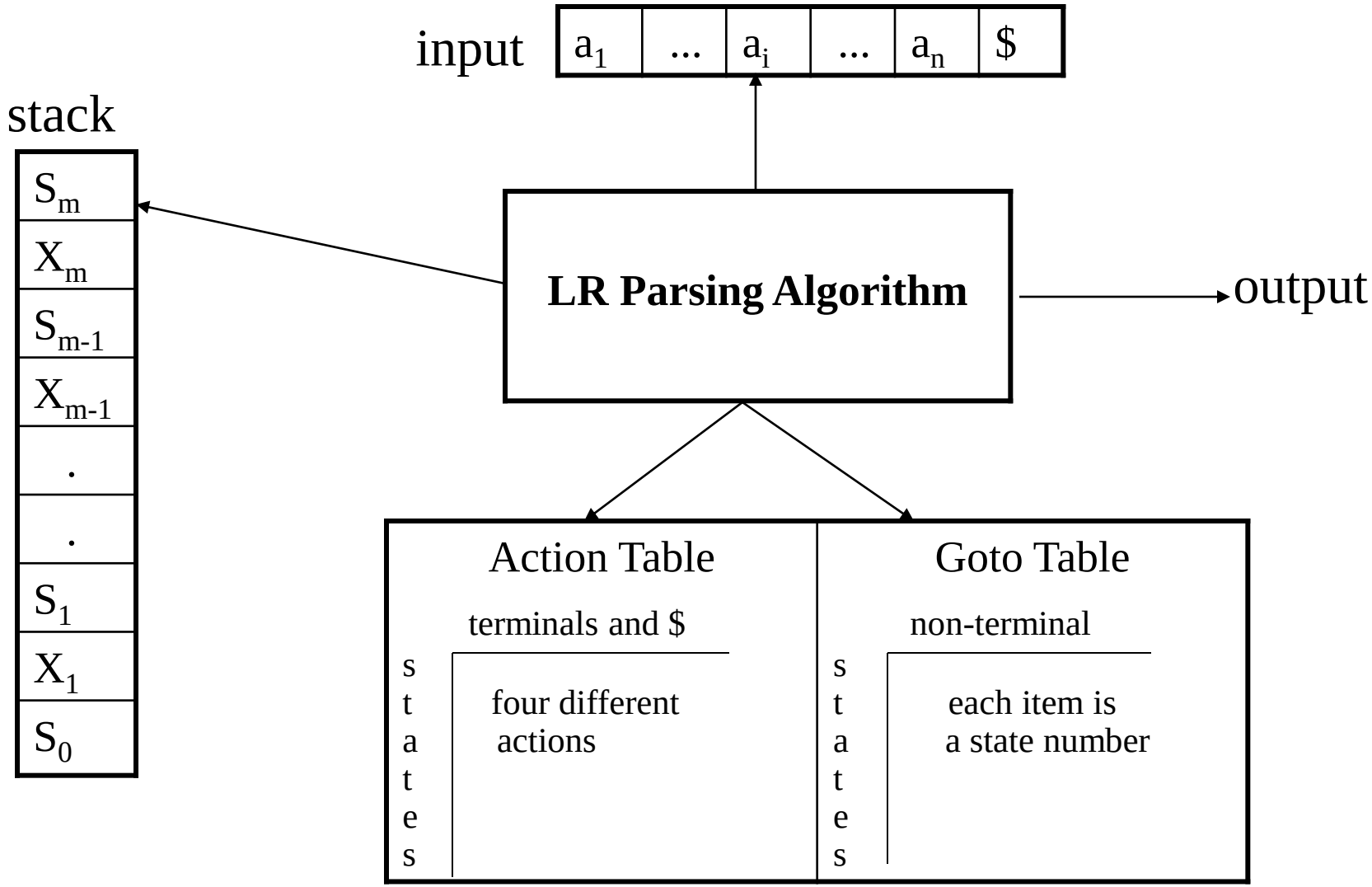
right-most derivation

k lookhead (k is omitted ➔ it is 1)

- LR parsing is attractive because:
  - LR parsing is most general non-backtracking shift-reduce parsing, yet it is still efficient.
  - The class of grammars that can be parsed using LR methods is a proper superset of the class of grammars that can be parsed with predictive parsers.

    LL(1)-Grammars $\subset$ LR(1)-Grammars
  - An LR-parser can detect a syntactic error as soon as it is possible to do so a left-to-right scan of the input.

LR Parsing Algorithm



LR Parsing Algorithm

```
set ip to point to the first symbol of w$;
repeat forever begin
    let s be the state on top of the stack and
        a the symbol pointed to by ip;
    if action[s, a] = shift s' then begin
        push a then s' on top of the stack;
        advance ip to the next input symbol
    end
    else if action[s, a] = reduce A → β then begin
        pop 2*|β| symbols off the stack;
        let s' be the state now on top of the stack;
        push A then goto[s', A] on top of the stack;
        output the production A → β
    end
    else if action[s, a] = accept then
        return
    else error()
end
```

A Configuration of LR Parsing Algorithm

- A configuration of a LR parsing is:

$$(\underline{S_o\ X_1\ S_1 \ldots X_m\ S_m},\ \underline{a_i\ a_{i+1} \ldots a_n\ \$})$$

Stack Rest of Input

- S_m and a_i decides the parser action by consulting the parsing action table. (*Initial Stack* contains just S_o)

- A configuration of a LR parsing represents the right sentential form:

$$X_1 \ldots X_m\ a_i\ a_{i+1} \ldots a_n\ \$$$

Actions of A LR-Parser

1. **shift s** -- shifts the next input symbol and the state **s** onto the stack

   $(S_o X_1 S_1 \ldots X_m S_m, a_i a_{i+1} \ldots a_n \$) \rightarrow (S_o X_1 S_1 \ldots X_m S_m a_i s, a_{i+1} \ldots a_n \$)$

2. **reduce A→β** (or **`rn`** where n is a production number)
   - pop 2|β| (=r) items from the stack;
   - then push **A** and **s** where **s=goto[s_{m-r},A] ---** after removing r many items from stack whatever is available state on top of stack

   $(S_o X_1 S_1 \ldots X_m S_m, a_i a_{i+1} \ldots a_n \$) \rightarrow (S_o X_1 S_1 \ldots X_{m-r} S_{m-r} A s, a_i \ldots a_n \$)$

   - Output is the reducing production reduce A→β

3. **Accept** – Parsing successfully completed

4. **Error** -- Parser detected an error (an empty entry in the action table)

Parsing Table

- It has two types of entries: ACTION, GOTO
- ACTION : two dimensional array indexed by state and terminals symbols
- GOTO : two dimensional array indexed by the state number and a Non-terminals

(SLR) Parsing Tables for Expression Grammar

1) $E \rightarrow E+T$
2) $E \rightarrow T$
3) $T \rightarrow T*F$
4) $T \rightarrow F$
5) $F \rightarrow (E)$
6) $F \rightarrow id$

	Action Table							Goto Table		
state	**id**	**+**	*****	**(**	**)**	**$**		**E**	**T**	**F**
0	s5			s4				1	2	3
1		s6				acc				
2		r2	s7		r2	r2				
3		r4	r4		r4	r4				
4	s5			s4				8	2	3
5		r6	r6		r6	r6				
6	s5			s4					9	3
7	s5			s4						10
8		s6			s11					
9		r1	s7		r1	r1				
10		r3	r3		r3	r3				
11		r5	r5		r5	r5				

Actions of A (S)LR-Parser -- Example

stack	input	action	output
0	id*id+id$	shift 5	
0id5	*id+id$	reduce by F→id	F→id
0F3	*id+id$	reduce by T→F	T→F
0T2	*id+id$	shift 7	
0T2*7	id+id$	shift 5	
0T2*7id5	+id$	reduce by F→id	F→id
0T2*7F10	+id$	reduce by T→T*F	T→T*F
0T2	+id$	reduce by E→T	E→T
0E1	+id$	shift 6	
0E1+6	id$	shift 5	
0E1+6id5	$	reduce by F→id	F→id
0E1+6F3	$	reduce by T→F	T→F
0E1+6T9	$	reduce by E→E+T	E→E+T
0E1	$	accept	

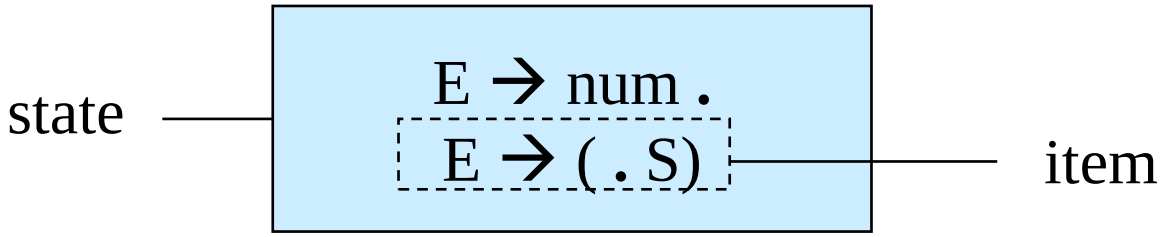
LR(0) Parsers

- Determine the actions without any lookahead
- Will help us understand shift-reduce parsing

- To build the parsing table:
  - Define states of the parser
  - Build a DFA to describe transitions between states
  - Use the DFA to build the parsing table
- Each LR(0) state is a set of LR(0) items
  - An LR(0) item: $X \rightarrow \alpha \,.\, \beta$ where $X \rightarrow \alpha\beta$ is a production in the grammar
  - The LR(0) items keep track of the progress on all of the possible upcoming productions
  - The item $X \rightarrow \alpha \,.\, \beta$ abstracts the fact that the parser already matched the string α at the top of the stack

Example LR(0) State

- An LR(0) item is a production from the language with a separator "**.**" somewhere in the RHS of the production



- Sub-string before "**.**" is already on the stack
- Sub-string after "**.**": what we might see next

LR(0) Items

- LR(0) item
  - A production with a dot at some position of the RHS

A ::= •XYZ	we are expecting XYZ
A ::= X •YZ	
A ::= XY •Z	
A ::= XYZ •	we have seen XYZ

Constructing LR(0) Parsing Tables – LR(0) Item

- An **LR(0) item** of a grammar G is a production of G a dot at the some position of the right side.
- Ex: $A \rightarrow aBb$ Possible LR(0) Items: $A \rightarrow \bullet aBb$
  (four different possibility) $A \rightarrow a \bullet Bb$
  $A \rightarrow aB \bullet b$
  $A \rightarrow aBb \bullet$
- Sets of LR(0) items will be the states of action and goto table of the SLR parser.
- A collection of sets of LR(0) items (**the canonical LR(0) collection**) is the basis for constructing SLR parsers.
- *Augmented Grammar*:

  G' is G with a new production rule $S' \rightarrow S$ where S' is the new starting symbol.

The Closure Operation

- If ***I*** is a set of LR(0) items for a grammar G, then ***closure(I)*** is the set of LR(0) items constructed from I by the two rules:
  1. Initially, every LR(0) item in I is added to closure(I).
  2. If $A \rightarrow \alpha.B\beta$ is in closure(I) and $B \rightarrow \gamma$ is a production rule of G; then $B \rightarrow .\gamma$ will be in the closure(I). We will apply this rule until no more new LR(0) items can be added to closure(I).

```
function closure ( I );
begin
    J := I;
    repeat
        for each item A → α·Bβ in J and each production
            B → γ of G such that B → ·γ is not in J do
                add B → ·γ to J
    until no more items can be added to J;
    return J
end
```

The Closure Operation -- Example

$E' \rightarrow E$

$E \rightarrow E+T$

$E \rightarrow T$

$T \rightarrow T*F$

$T \rightarrow F$

$F \rightarrow (E)$

$F \rightarrow id$

closure($\{E' \rightarrow \bullet E\}$) =

$\{ \; E' \rightarrow \bullet E$ ← kernel items

$E \rightarrow \bullet E+T$

$E \rightarrow \bullet T$

$T \rightarrow \bullet T*F$

$T \rightarrow \bullet F$

$F \rightarrow \bullet (E)$

$F \rightarrow \bullet id \; \}$

Goto Operation

- If I is a set of LR(0) items and X is a grammar symbol (terminal or non-terminal), then goto(I,X) is defined as follows:
  - If $A \rightarrow \alpha.X\beta$ in I then every item in **closure({A → αX.β})** will be in goto(I,X).

Example:

I ={ E' → .E, E → .E+T,
E → .T, T → .T*F, T → .F,
F → .(E), F → .id }

goto(I,E) = { E' → E., E → E.+T }

goto(I,T) = { E → T., T → T.*F }

goto(I,F) = {T → F. }

goto(I,() = { F → (.E), E → .E+T,
E → .T, T → .T*F, T → .F,
F → .(E), F → .id }

goto(I,id) = { F → id. }

```
procedure items(G');
begin
    C := {closure({[S' → ·S]})};
    repeat
        for each set of items I in C and each grammar symbol X
            such that goto(I, X) is not empty and not in C do
                add goto(I, X) to C
    until no more sets of items can be added to C
end
```

Construction of The Canonical LR(0) Collection

- ***Algorithm***:

  C is { closure({S'→ • S}) }

  repeat the followings until no more set of LR(0) items can be added to ***C***.

  for each I in ***C*** and each grammar symbol X

  if goto(I,X) is not empty and not in ***C***

  add goto(I,X) to ***C***

- goto function is a DFA on the sets in C.
- To create the SLR parsing tables for a grammar G, we will create the canonical LR(0) collection of the grammar G'.

Start State and Closure

- Start state
  - Augment grammar with production: S' → S $
  - Start state of DFA has empty stack: S' → **.** S $
- Closure of a parser state:
  - Start with Closure(S) = S
  - Then for each item in S:
    - X → α **.** Y β
    - Add items for all the productions Y → γ to the closure of S: Y → **.** γ

Closure

$S \rightarrow (L) \mid id$
$L \rightarrow S \mid L,S$

DFA start state

$S' \rightarrow \mathbf{.}\ S\ \$$

closure

$S' \rightarrow \mathbf{.}\ S\ \$$
$S \rightarrow \mathbf{.}\ (L)$
$S \rightarrow \mathbf{.}\ id$

- Set of possible productions to be reduced next
- Closure of a parser state, S:
  - Start with Closure(S) = S
  - Then for each item in S:
    - $X \rightarrow \alpha\ \mathbf{.}\ Y\ \beta$
    - Add items for all the productions $Y \rightarrow \gamma$ to the closure of S: $Y \rightarrow \mathbf{.}\ \gamma$

The Goto Operation

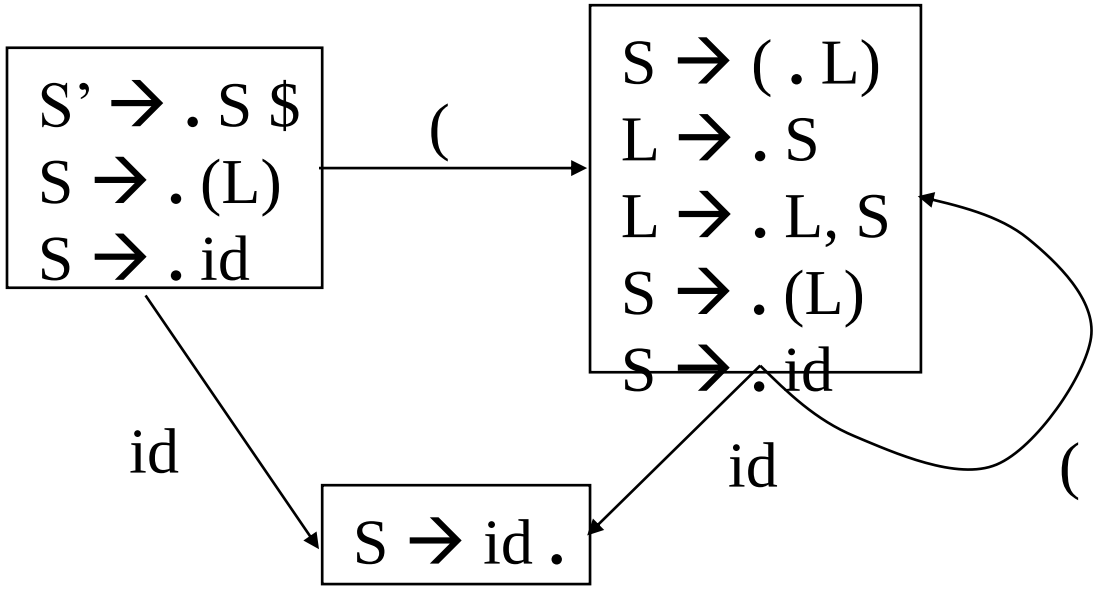
- Goto operation = describes transitions between parser states, which are sets of items
- Algorithm: for state S and a symbol Y
  - If the item [X → α . Y β] is in I, then
  - Goto(I, Y) = Closure([X → α Y . β])

S' → . S $
S → . (L)
S → . id

Goto(S, '(')

Closure({ S → (. L) })

Goto: Terminal Symbols



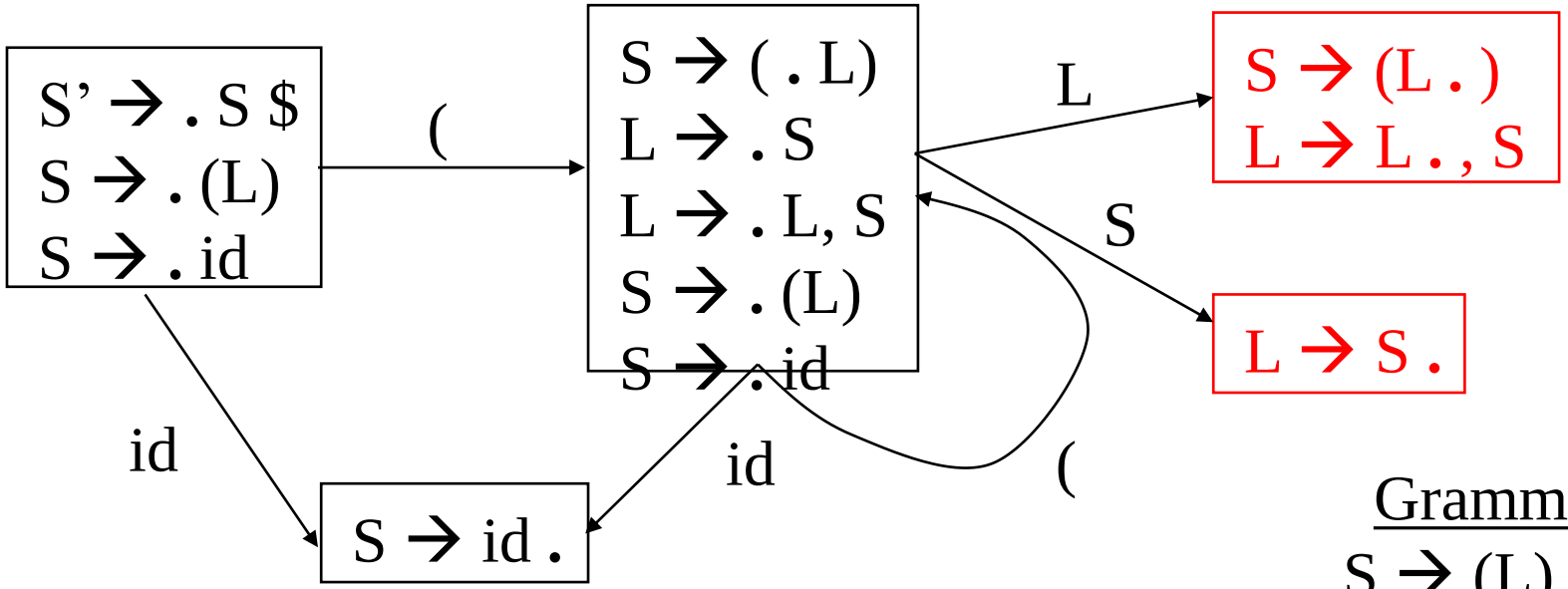
Grammar

S → (L) | id

L → S | L,S

In new state, include all items that have appropriate input symbol just after dot, advance do in those items and take closure

Goto: Non-terminal Symbols



Grammar

S → (L) | id

L → S | L,S

same algorithm for transitions on non-terminals

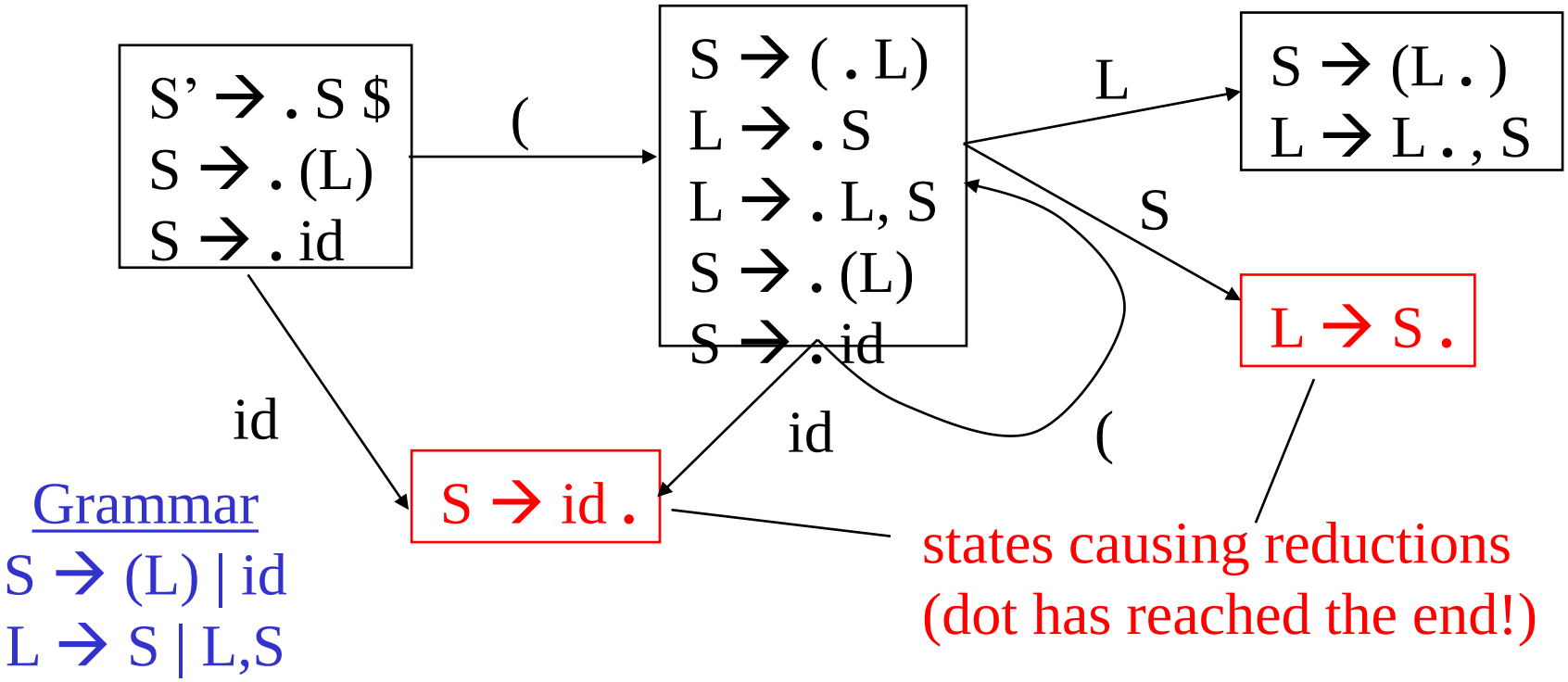
Example

$E' \rightarrow E$
$E \rightarrow E + T \mid T$
$T \rightarrow T * F \mid F$
$F \rightarrow (E) \mid id$

1. If I = { [E' → **.** E]}, then Closure(I) = ??

2. If I = { [E' → E .], [E → E . + T] }, then Goto(I,+) = ??

Applying Reduce Actions



Grammar

S → (L) | id

L → S | L,S

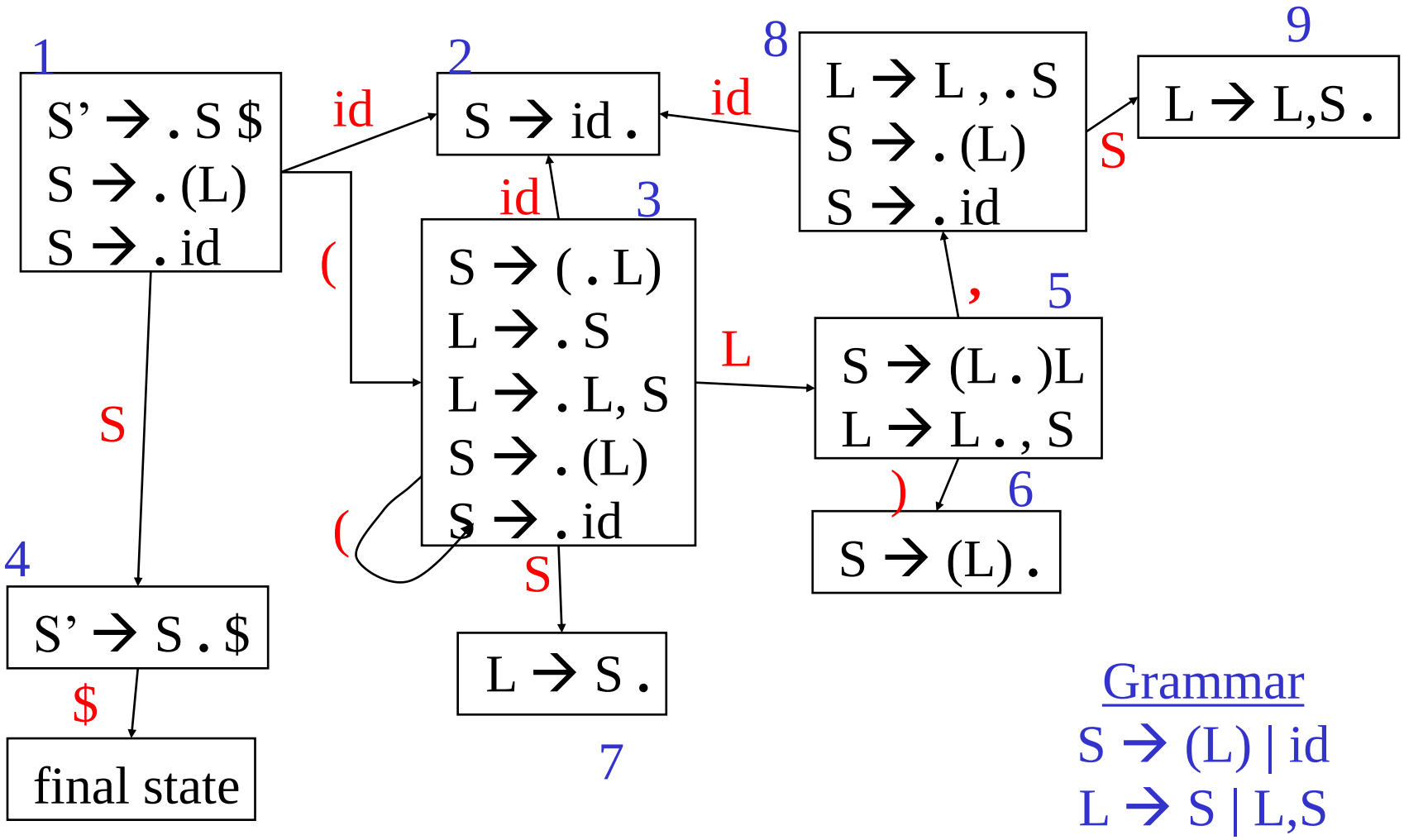
Pop RHS off stack, replace with LHS X (X → β), then rerun DFA (e.g., (x))

Reductions

- On reducing X → β with stack αβ
  - Pop β off stack, revealing prefix α and state
  - Take single step in DFA from top state
  - Push X onto stack with new DFA state
- Example

derivation	stack	input	action
((a),b) ←	1 (3 (3	a),b)	shift, goto 2
((a),b) ←	1 (3 (3 a 2	),b)	reduce S → id
((S),b) ←	1 (3 (3 S 7	),b)	reduce L → S

Full DFA



Grammar

S → (L) | id

L → S | L,S

Parsing Example ((a),b)

derivation	stack	input	action
((a),b) ←	1	((a),b)	shift, goto 3
((a),b) ←	1(3	(a),b)	shift, goto 3
((a),b) ←	1(3(3	a),b)	shift, goto 2
((a),b) ←	1(3(3a2	),b)	reduce S→id
((S),b) ←	1(3(3(S7	),b)	reduce L→S
((L),b) ←	1(3(3(L5	),b)	shift, goto 6
((L),b) ←	1(3(3L5)6	,b)	reduce S→(L)
(S,b) ←	1(3S7	,b)	reduce L→S
(L,b) ←	1(3L5	,b)	shift, goto 8
(L,b) ←	1(3L5,8	b)	shift, goto 9
(L,b) ←	1(3L5,8b2	)	reduce S→id
(L,S) ←	1(3L8,S9	)	reduce L→L,S
(L) ←	1(3L5	)	shift, goto 6
(L) ←	1(3L5)6		reduce S→(L)
S ←	1S4	$	done

S → (L) | id
L → S | L,S

Building the Parsing Table

- States in the table = states in the DFA
- For transition S → S' on terminal C:
  - Table[S,C] += Shift(S')
- For transition S → S' on non-terminal N:
  - Table[S,N] += Goto(S')
- If S is a reduction state X → β then:
  - Table[S,*] += Reduce(X → β)

Computed LR(0) Parsing Table

State	Input terminal					Non-terminals	
	(	)	id	,	$	S	L
1	s3		s2			g4	
2	S→id	S→id	S→id	S→id	S→id		
3	s3		s2			g7	g5
4					accept		
5		s6		s8			
6	S→(L)	S→(L)	S→(L)	S→(L)	S→(L)		
7	L→S	L→S	L→S	L→S	L→S		
8	s3		s2			g9	
9	L→L,S	L→L,S	L→L,S	L→L,S	L→L,S		

blue = shift red = reduce

LR(0) Summary

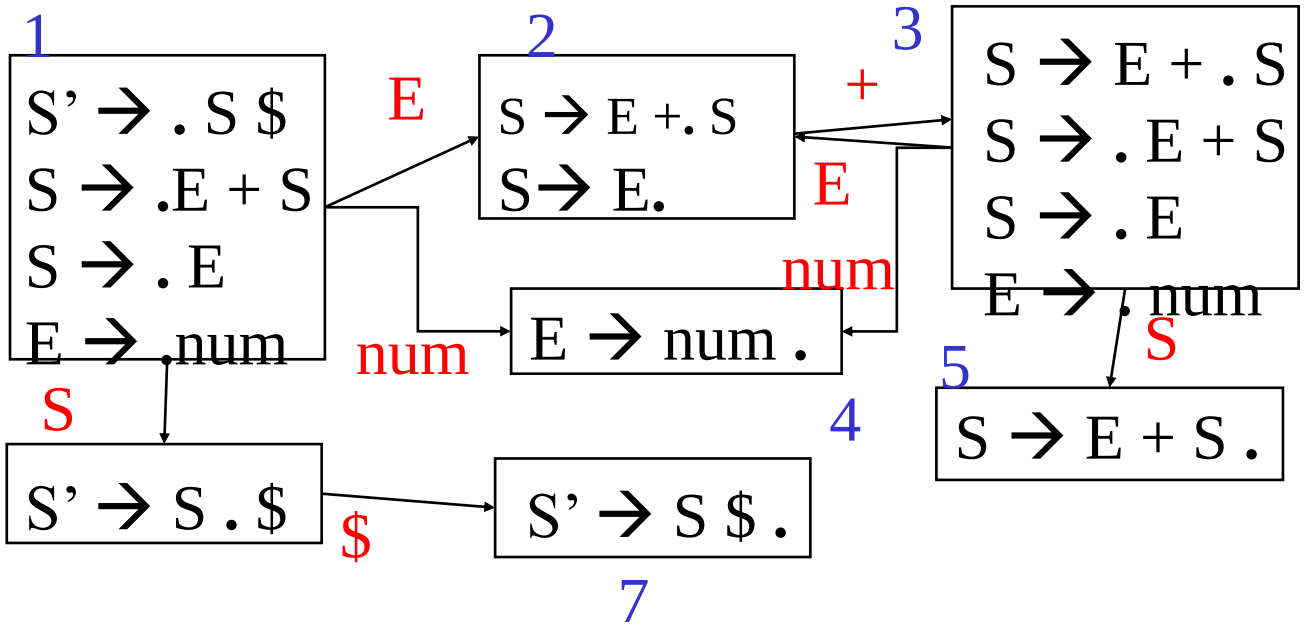
- LR(0) parsing recipe:
  - Start with LR(0) grammar
  - Compute LR(0) states and build DFA:
    - Use the closure operation to compute states
    - Use the goto operation to compute transitions
  - Build the LR(0) parsing table from the DFA

Example

Generate the DFA for the following grammar of addition of numbers

$S \rightarrow E + S \mid E$
$E \rightarrow num$

LR(0) Parsing Table



Grammar

S → E + S | E

E → num

Shift or reduce in state 2?

	num	+	$	E	S
1	s4			g2	g6
2	S→E	s3/S→E	S→E		

Conflicts During Shift-Reduce Parsing

- There are context-free grammars for which shift-reduce parsers cannot be used.
- Stack contents and the next input symbol may not decide action:
  - **shift/reduce conflict**: Whether make a shift operation or a reduction.
  - **reduce/reduce conflict**: The parser cannot decide which of several reductions to make.

- An ambiguous grammar can never be a LR grammar.

- Grammar for addition of numbers
  - $S \rightarrow S + E \mid E$
  - $E \rightarrow num$
- Left-associative version is LR(0)
- While Right-associative is not LR(0) as seen.
  - $S \rightarrow E + S \mid E$
  - $E \rightarrow num$

LR(0) Limitations

- An LR(0) machine only works if states with reduce actions have a single reduce action
  - Always reduce regardless of lookahead
- With a more complex grammar, construction gives states with shift/reduce or reduce/reduce conflicts
- Need to use lookahead to choose

OK

$L \rightarrow L\ ,\ S\ .$

shift/reduce

$L \rightarrow L\ ,\ S\ .$
$S \rightarrow S\ .\ ,\ L$

reduce/reduce

$L \rightarrow S\ ,\ L\ .$
$L \rightarrow S\ .$

Solve Conflict With Lookahead

- 3 popular techniques for employing lookahead of 1 symbol with bottom-up parsing
  - SLR – Simple LR
  - LALR – LookAhead LR
  - LR(1)
- Each as a different means of utilizing the lookahead
  - Results in different processing capabilities

Constructing SLR Parsing Table

(of an augumented grammar G')

1. Construct the canonical collection of sets of LR(0) items for G'.
   $C \leftarrow \{I_0,\ldots,I_n\}$

2. Create the parsing action table as follows
   - If a is a terminal, $A \rightarrow \alpha.a\beta$ in I_i and $goto(I_i,a)=I_j$ then action[i,a] is ***shift j.***
   - If $A \rightarrow \alpha.$ is in I_i, then action[i,a] is ***reduce*** $\boldsymbol{A \rightarrow \alpha}$ for all a in FOLLOW(A) where $A \neq S'$.(see example $E \rightarrow T.$)
   - If $S' \rightarrow S.$ is in I_i, then action[i,$] is ***accept***.
   - If any conflicting actions generated by these rules, the grammar is not SLR(1).

3. Create the parsing goto table
   - for all non-terminals A, if $goto(I_i,A)=I_j$ then goto[i,A]=j

4. All entries not defined by (2) and (3) are errors.

5. Initial state of the parser contains $S' \rightarrow .S$

SLR Parsing

- SLR Parsing = Easy extension of LR(0)
  - For each reduction X → β, look at next symbol C
  - Apply reduction only if <u>C is in FOLLOW(X)</u>
- SLR parsing table eliminates some conflicts
  - Same as LR(0) table except reduction rows
  - Adds reductions X → β only in the columns of symbols in FOLLOW(X)

Example: FOLLOW(S) = {$}

<u>Grammar</u>
S → E + S | E
E → num

	num	+	$	E	S
1	s4			g2	g6
2		s3	S→E		

SLR Parsing Table

- Reductions do not fill entire rows as before
- Otherwise, same as LR(0)

Grammar

S → E + S | E

E → num

	num	+	$	E	S
1	s4			g2	g6
2		s3	S→E		
3	s4			g2	g5
4		E→num	E→num		
5			S→E+S		
6			s7		
7			accept		

Example-1

Consider:

$S \rightarrow L = R$

$S \rightarrow R$

$L \rightarrow *R$

$L \rightarrow ident$

$R \rightarrow L$

Think of L as l-value, R as r-value, and * as a pointer dereference

Cont.

$S \rightarrow L{=}R$
$S \rightarrow R$
$L \rightarrow {*}R$
$L \rightarrow id$
$R \rightarrow L$

I_0: $S' \rightarrow .S$
$S \rightarrow .L{=}R$
$S \rightarrow .R$
$L \rightarrow .{*}R$
$L \rightarrow .id$
$R \rightarrow .L$

I_1: $S' \rightarrow S.$

I_2: $S \rightarrow L.{=}R$
$R \rightarrow L.$

I_3: $S \rightarrow R.$

I_4: $L \rightarrow {*}.R$
$R \rightarrow .L$
$L \rightarrow .{*}R$
$L \rightarrow .id$

I_5: $L \rightarrow id.$

I_6: $S \rightarrow L{=}.R$
$R \rightarrow .L$
$L \rightarrow .{*}R$
$L \rightarrow .id$

I_7: $L \rightarrow {*}R.$

I_8: $R \rightarrow L.$

I_9: $S \rightarrow L{=}R.$

Problem

FOLLOW(R)={=,$}

= → shift 6
→ reduce by $R \rightarrow L$

shift/reduce conflict

Not SLR(1)

Example-2

$S \rightarrow AaAb$

$S \rightarrow BbBa$

$A \rightarrow \varepsilon$

$B \rightarrow \varepsilon$

I_0: $S' \rightarrow .S$

$S \rightarrow .AaAb$

$S \rightarrow .BbBa$

$A \rightarrow .$

$B \rightarrow .$

Problem

FOLLOW(A)={a,b}

FOLLOW(B)={a,b}

a → reduce by $A \rightarrow \varepsilon$

a → reduce by $B \rightarrow \varepsilon$

reduce/reduce conflict

b → reduce by $A \rightarrow \varepsilon$

b → reduce by $B \rightarrow \varepsilon$

reduce/reduce conflict

Not SLR(1)

Example-3

I_0: E' → .E
E → .E+T
E → .T
T → .T*F
T → .F
F → .(E)
F → .id

I_1: E' → E.
E → E.+T

I_2: E → T.
T → T.*F

I_3: T → F.

I_4: F → (.E)
E → .E+T
E → .T
T → .T*F
T → .F
F → .(E)
F → .id

I_5: F → id.

I_6: E → E+.T
E → .T
T → .T*F
T → .F
F → .(E)
F → .id

I_7: T → T*.F
F → .(E)
F → .id

I_8: F → (E.)
E → E.+T

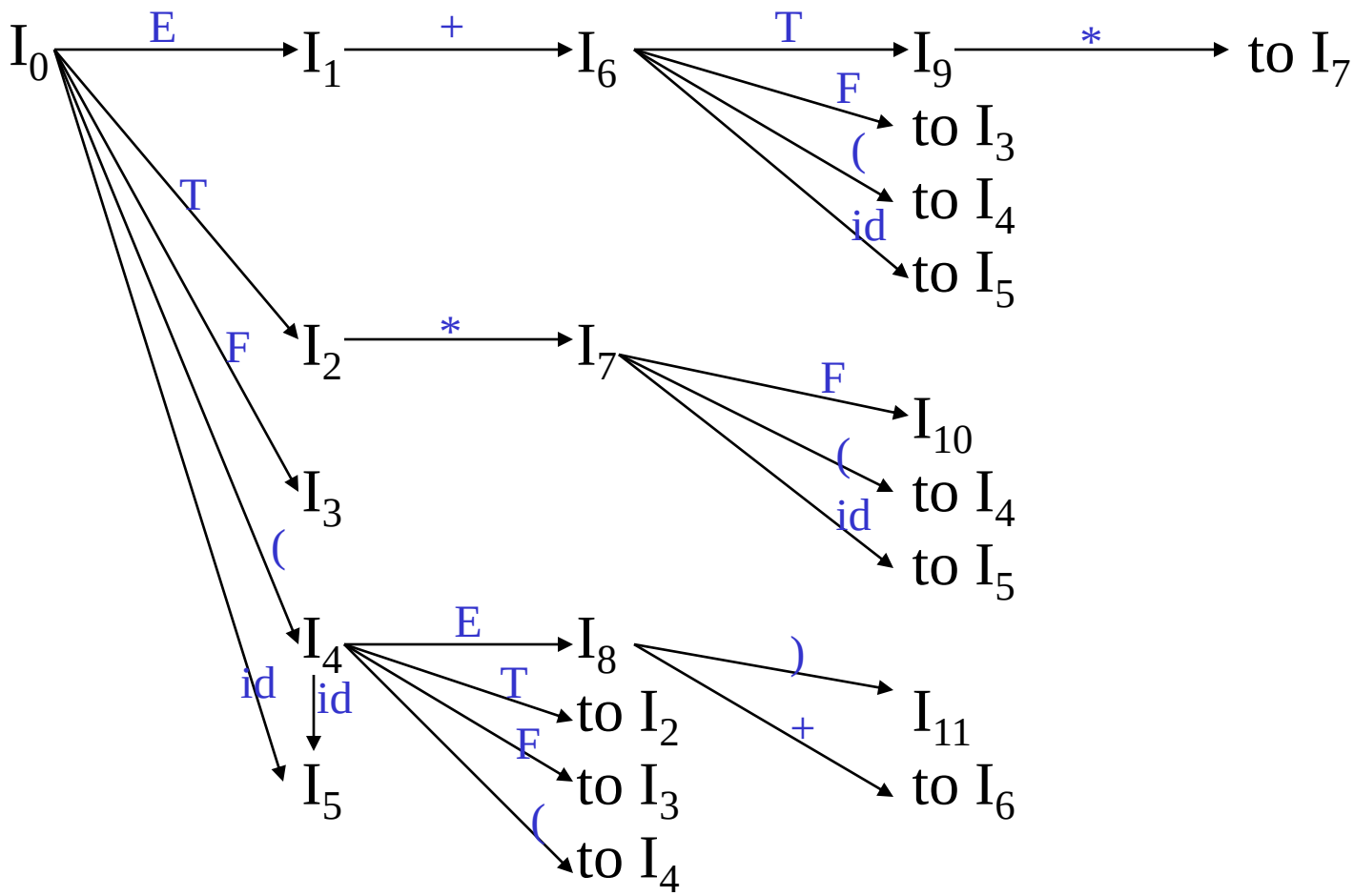
I_9: E → E+T.
T → T.*F

I_{10}: T → T*F.

I_{11}: F → (E).

Canonical collection of LR(0)

Transition Diagram (DFA) of Goto Function



Parsing Tables of Expression Grammar

Action Table Goto Table

state	id	+	*	(	)	$		E	T	F
0	s5			s4				1	2	3
1		s6				acc				
2		r2	s7		r2	r2				
3		r4	r4		r4	r4				
4	s5			s4				8	2	3
5		r6	r6		r6	r6				
6	s5			s4					9	3
7	s5			s4						10
8		s6			s11					
9		r1	s7		r1	r1				
10		r3	r3		r3	r3				
11		r5	r5		r5	r5				

SLR(1)???

SLR(1) Grammar

- An LR parser using SLR(1) parsing tables for a grammar G is called as the SLR(1) parser for G.
- If a grammar G has an SLR(1) parsing table, it is called SLR(1) grammar (or SLR grammar in short).
- Every SLR grammar is unambiguous, but every unambiguous grammar is not a SLR grammar.

shift/reduce and reduce/reduce conflicts

- If a state does not know whether it will make a shift operation or reduction for a terminal, we say that there is a **shift/reduce conflict**.

- If a state does not know whether it will make a reduction operation using the production rule `i` or `j` for a terminal, we say that there is a **reduce/reduce conflict**.

- If the SLR parsing table of a grammar G has a conflict, we say that that grammar is not SLR grammar.

LR(0) and SLR(1) Parsers

- SLR(1) Parse table= LR(0) parse table + **Follow** (for ex to find follow of T in the case of T->E. item)

Weaknesses of SLR(1)

- Using the only FOLLOW sets to drive reduce decisions throws away useful informations.
- Tokens in a FOLLOW set arise from different productions, but they get lumped together in the FOLLOW set with tokens arising from other productions.
- When it comes time to decide on a reduce move, we sometimes need to be more specific, and associate the look ahead with the particular production that added that token to the FOLLOW set.